

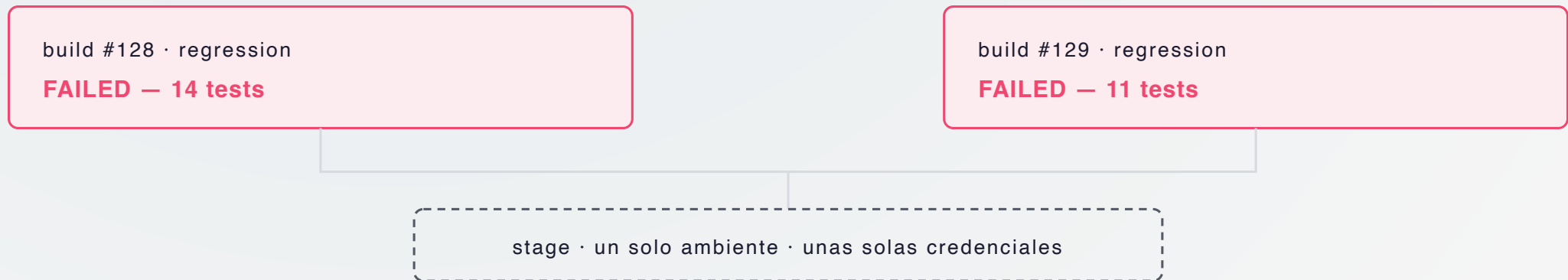
# Tu setup de QA no se diseña: se cultiva

De prompt suelto a skills, rules y hooks.

Edgardo Crovetto · 2026

# Un día perdí 90 minutos

cazando flaky tests fantasma



Se pisaban entre sí, y nadie lo sabía. Todo **rojo**. Nada roto.

**Hoy eso no puede volver a pasarme.**

**Y no es porque yo me acuerde: es porque mi setup se acuerda por mí.**

Esta charla es la historia de cómo llegué ahí.

## ■ Quién soy

**Edgardo Crovetto** · Senior QA automation engineer

Java + TypeScript · tests automatizados · CI/CD · pipelines

**Me gusta mejorar procesos — sobre todo los que ya estaban funcionando.**

“ Esta charla no es sobre features. Es sobre **un patrón que se repite** — y lo que hacés con eso. ”

[linkedin.com/in/edgardocrovetto](https://linkedin.com/in/edgardocrovetto)

## ■ La pregunta de hoy

| ¿Puedo automatizar mi proceso de QA con IA?

Spoiler: sí — **sobre las bases que ya tenés.**

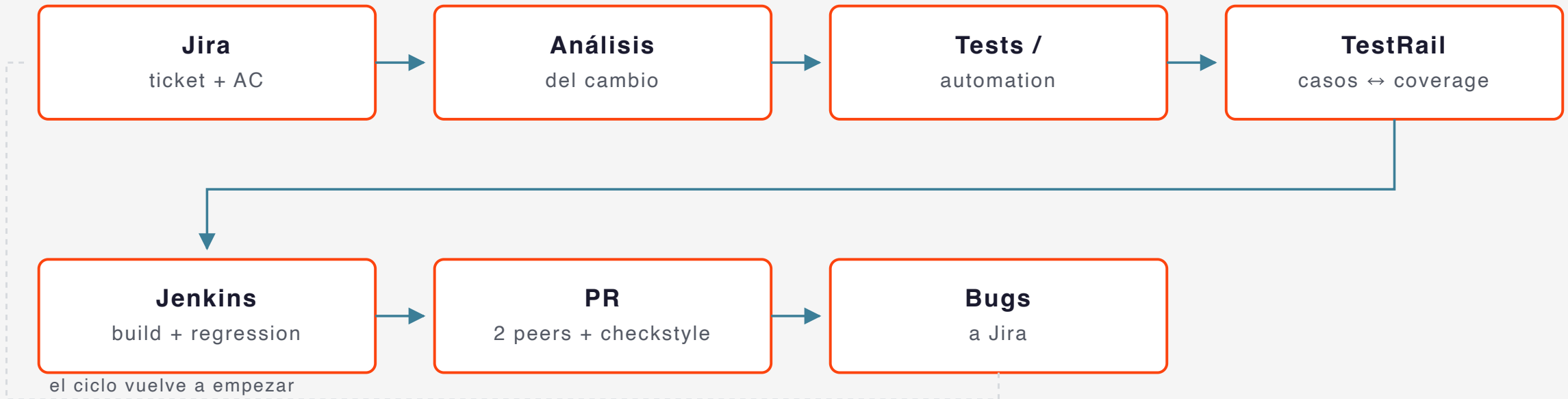


## ■ Mi setup antes de IA

Plataforma de streaming · servicio backend REST

**Framework:** Java + TestNG + RestAssured + Allure reports + Jira + TestRail + Jenkins

**Pipeline diario:**



“ Funcionaba. Pero todo el conocimiento de cada ticket vivía **en mi cabeza.** ”

## ■ Mi día como QA, antes de todo esto

- **Análisis del ticket** — Jira + repo + Confluence (AC) + TestRail · nada de eso queda escrito en un solo lugar
- **Mapeo AC ↔ cambio** — comparo docs vs branch a ojo, página por página
- **Test cases + automation** — copio AC a TestRail, traduzco a TestNG/RestAssured, linkeo IDs a mano
- **Ejecución multi-env** — Jenkins contra 3 ambientes · comparo resultados · investigo cada **rojo**
- **PR review** — armo la evidencia, pingeo 2 peers, espero, re-pingeo
- **Bug encontrado** — abro ticket, pego logs, follow-up del ciclo de vida en Jira
- **Mañana** — sesión nueva. Re-explico el ticket, el plan, las convenciones.

**Mucho de eso es conocimiento que se pierde entre sesiones.**

¿Y si ese conocimiento sobreviviera?

**La tesis:**  
**Mi setup no se diseña,**  
**se cultiva.**



## ■ Mis primeros pasos con Claude

### 1. Conectar Claude al repo del trabajo

- Todo arrancó con un `CLAUDE.md` de apenas 5 líneas — se lo pedís en la conversación, no lo escribís a mano
- Lo demás vino con el tiempo

### 2. Agregar MCPs, uno a uno

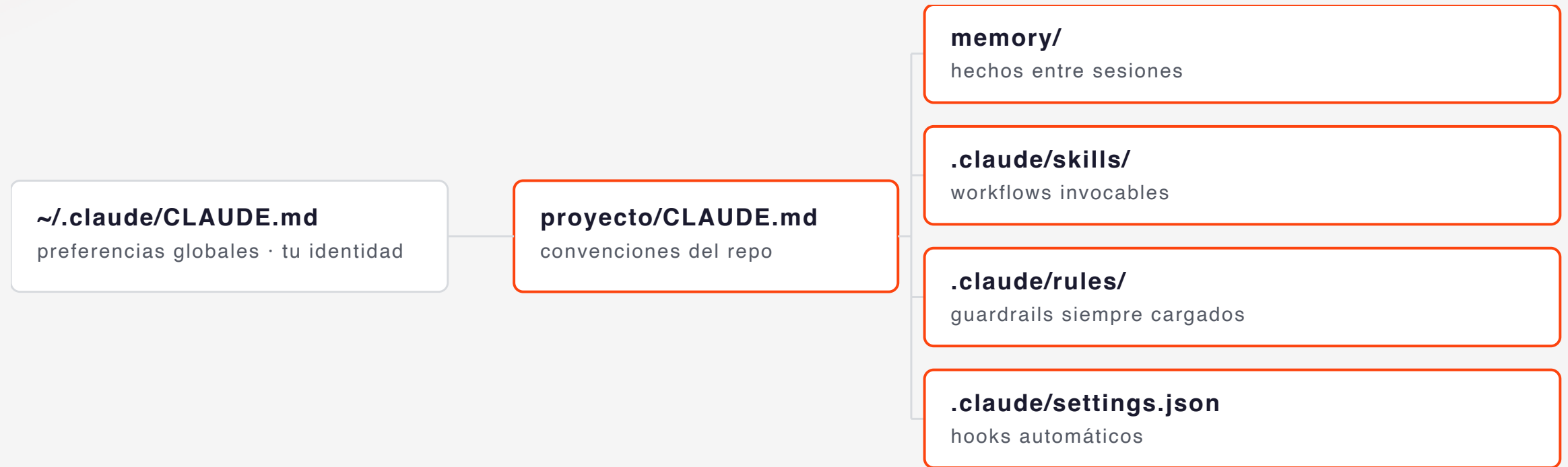
Jira, TestRail, Jenkins, GitHub, Confluence — el detalle, enseguida.

A partir de ahí: **prompts del día a día.**

No hubo plan. Aparecieron patrones.



## ■ Cómo Claude conoce mi proyecto



Todo es **texto plano en disco**. Versionado. Reviewable.

# MCPs — los puentes a sistemas externos

Memory, skills, rules y hooks viven en tu repo.  
El trabajo real cruza varios sistemas. Los MCPs los conectan.

*Conectar uno también se pide en conversación — no hay instalador que armar a mano.*

| MCP                                                                                                 | PARA QUÉ LO USO                                         |
|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------|
|  <b>Jira</b>       | Leer ticket + AC, comentar, transicionar el estado      |
|  <b>TestRail</b>   | Leer/escribir test cases, asociar runs a builds         |
|  <b>Jenkins</b>    | Triggerear builds, leer resultados, descargar logs      |
|  <b>GitHub</b>     | Leer el PR y su diff, postear el review, ver los checks |
|  <b>Confluence</b> | Leer specs y acceptance criteria                        |

Una sola conversación con Claude puede pasar por los 5.  
*El review de la Demo 4 se postea por acá.*

## ■ La pirámide de promoción

**Una sola historia:** el typecheck que me olvidaba después de cada rebase, subiendo un nivel cada vez que el anterior no alcanzó.

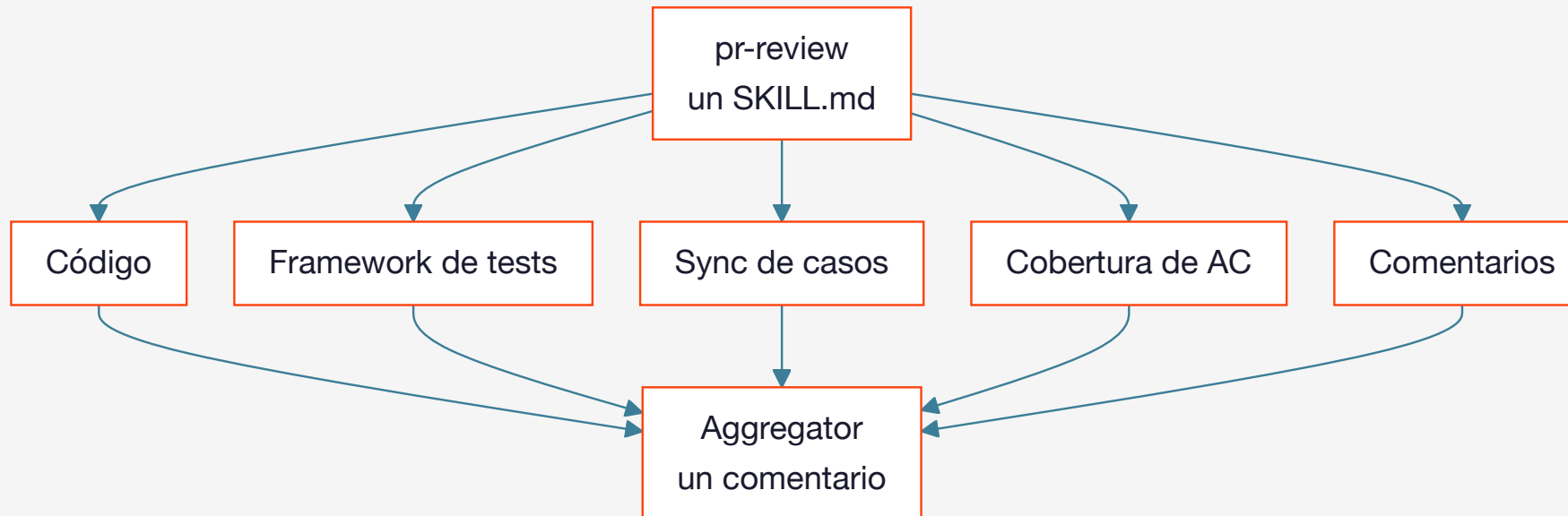


**Saltea la Rule** — y no es olvido: una rule me lo *recuerda*, y el problema era justamente que recordármelo no alcanzaba. **Pocas piezas llegan hasta arriba:** no es una escalera que subís entera, es un menú. La Rule tiene su propia cicatriz — `no-parallel-ci`, los 90 minutos del principio.

“ Ninguna pieza se diseñó. Cada una fue admitir que acordarse no escala. ”

## ■ Un skill no siempre ejecuta: a veces delega

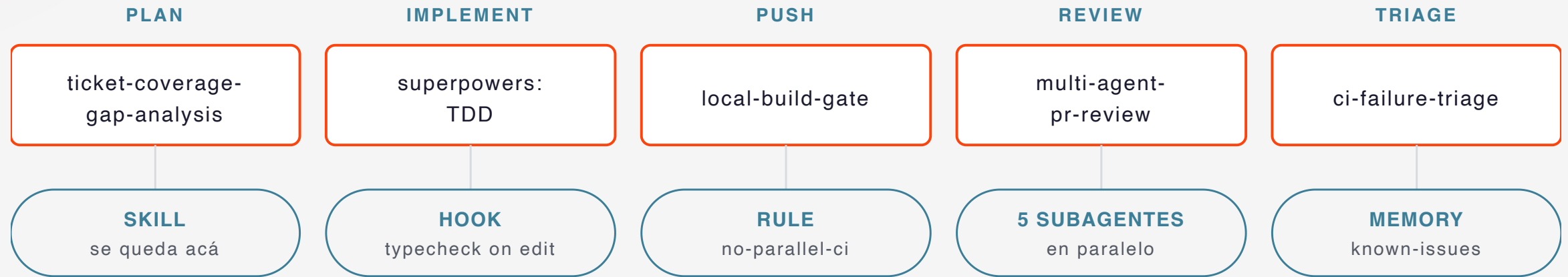
El skill de review es un `SKILL.md` como cualquier otro. Lo distinto es lo que hace adentro: en vez de correr pasos él mismo, **despacha subagentes** — cada uno con su contexto propio: no ve tu historial, y al principal solo vuelve el resumen.



**Antes de que un peer humano vea el PR, ya pasó por 5 revisores especializados** — y son **míos**: saben que el id del caso es el prefijo del método y qué AC pedía el ticket. Eso no lo sabe ningún plugin.

**Lo que no cambia: quien aprueba sigue siendo responsable de lo que aprueba.**

## ■ El flujo end-to-end



rules y memory importadas: siempre cargadas · el registry se lee cuando hace falta

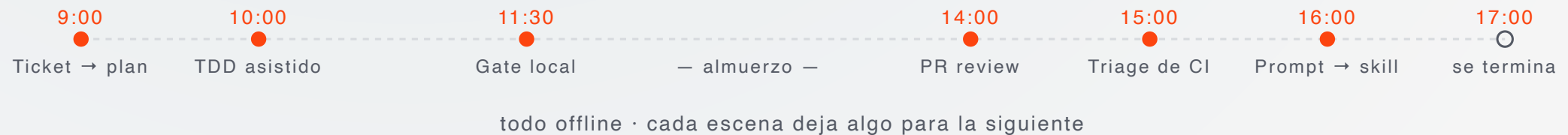
↺ corrección repetida 3x —> writing-skills —> skill nueva

**Cada pieza compone con las demás.** No hay un workflow único — hay piezas.

# Ahora, en vivo

Un día de QA, del ticket al merge

6 escenas a lo largo de una jornada de 8 horas · repo `claude-qa-demo`



*El demo está en TypeScript para que entre en pantalla y compile rápido.*

*El patrón es **idéntico** en Java/TestNG/RestAssured.*

## ■ Demo 1 · 9:00 — Ticket → plan de cobertura

| “ ↩ Resuelve: las 4 pestañas para armar el panorama completo. ”

**Input:** un ticket de Jira mockeado en `mocks/jira/DEMO-100.json` .

**Prompt:**

| “ "Planificá el trabajo para DEMO-100." ”

**Skill invocada:** `ticket-coverage-gap-analysis`

**Output:**

- Mapa de cobertura existente (tabla)
- Gaps identificados (✅ / ⚠ / ❌)
- TodoWrite con casos propuestos + estimación

*El plan queda con un ❌: slug malformado sin cubrir. →*



## ■ Demo 2 · 10:00 — TDD asistido

| “ ↩ Resuelve: saltar el "red" y aterrizar directo en código sin test que lo respalde. ”

Demo 1 dejó un **✗**: **DEMO-100 pide que un slug malformado sea rechazado.**

Hoy `getChannelBySlug('News Channel!')` devuelve `null`, como si no existiera.

### Prompt:

| “ "Cerrá ese gap con TDD." ”

**Skill invocada:** `superpowers:test-driven-development`

(descargada del marketplace, no la escribí yo)

1. Test que espera el rechazo → **red**
2. Validación mínima del formato → **green**
3. Refactor opcional

El skill guía el orden — y hoy lo respetó.

Gap cerrado, tests **verdes**. Ahora quiero correr esto en Cl. →

## ■ Demo 3 · 11:30 — Gate local antes de CI

“ ↩ Resuelve: pushear un cambio con un typo y enterarte 10 minutos después, cuando Jenkins ya arrancó el build. ”

### Prompt:

“ "Triggeá un build de Jenkins para esta branch." ”

Lo que pasa en vivo:

1. `local-build-gate` → typecheck + tests locales ✅
2. `no-parallel-ci.mdc` → **build 43 ya está corriendo en stage** ( `build-43-running.json` )
3. Claude **se niega a triggerear**: esperar ~12 min o cambiar de env

**El guardrail no me cuida a mí del agente — nos cuida a los dos del incidente.**

Los 90 minutos del principio, exactamente.

*Build 43 termina, corre el mío, abro el PR. →*

## ■ Demo 4 (1/2) · 14:00 — Multi-agent PR review 🌟

| “ ↩ Resuelve: repetir los mismos comentarios review tras review. ”

PR sembrado con 5 bugs distintos ( `mocks/github/pr-7.diff` ):

- `silent-failure-hunter` → `catch (e) { return [] }` se traga errores
- `type-design-analyzer` → `as Channel` miente
- `comment-analyzer` → comentario que dice "sorted" pero no ordena
- `pr-test-analyzer` → test que solo verifica `toBeDefined()`
- `code-reviewer` → `// TODO` en producción

**5 en paralelo, un solo mensaje.** Acá corren los **genéricos** de un plugin:  
la versión simplificada del demo, para que ande en cualquier repo.

## ■ Demo 4 (2/2) — el resultado agregado

claude · comentó en #7 · hace 1 minuto

### Review summary

**BLOCKER** `catch (e) { return [] }` — un fallo de red vuelve como lista vacía fallas silenciosas

**SUGGESTION** `as Channel` afirma un tipo que puede no existir: devuelve **undefined** diseño de tipos

**SUGGESTION** el comentario promete **sorted by relevance** — la función no ordena comentarios

**SUGGESTION** el único test nuevo asserta **toBeDefined()**: pasa con el channel equivocado diseño de tests

**NITPICK** `// TODO: should probably be an enum` quedó en producción código

► [Per-axis details](#)

5× paralelo, contexto aislado, 1 comentario al final.

**Lo leo entero antes de postearlo. Si alguien pregunta por qué se bloqueó el PR, la respuesta soy yo — "no sé, lo hizo la IA" no es una respuesta.**

## ■ ¿Y esto no lo hacía ya un linter?

Buena parte sí — y **el linter no se saca**: es más barato y no se cansa.

De las 5 cosas del reporte, un linter agarra **dos**: el `catch` que se traga la excepción ( `S2486` ) y el `// TODO` ( `S1135` ). Las otras tres —el comentario que miente, el cast que miente, el test que no prueba nada— hay que **leerlas**.

**El linter matchea patrones. El subagente lee.** Van juntos.

*Review adentro. Antes del merge, la regression completa en Jenkins. →*

■ Demo 5 · 15:00 — Triage de fallas de CI

| “ ↩ Resuelve: triagear tests **rojos** a mano contra known-issues. ”

**Input:** `mocks/jenkins/build-44.json` — 5 **rojos** sin etiquetar: nombre, mensaje, stack y los commits. Y es la misma branch de hoy: `feature/DEM0-100-channels-coverage` .

**Skill invocada:** `ci-failure-triage` + registry `memory/known-issues.md`

| CATEGORÍA           | DE DÓNDE SALE                                                                                  |
|---------------------|------------------------------------------------------------------------------------------------|
| Flaky test conocido | La firma matchea el registry (2× <code>Test timed out in 5000ms</code> , visto hasta build 39) |
| Infra               | El ambiente no responde ( <code>ECONNREFUSED</code> a auth)                                    |
| Regresión real      | No está en el registry y el commit toca código relacionado                                     |

**La categoría se deduce de la evidencia — no viene dada en el JSON.**

## ■ El triage no termina en la categoría

Son 5 fallas para que entren en pantalla. En un build de cientos, el mismo skill lee los **logs completos**, agrupa las fallas relacionadas entre sí y te dice **por dónde empezar a mirar** — no solo "esto es un flaky test".



*Tres veces hoy le pedí lo mismo: "chequeá el registry primero". →*

## ■ Demo 6 · 16:00 — De prompt repetido a skill ✨

| “ ↩ Resuelve: re-explicar el ticket, el plan, las convenciones en una sesión nueva. ”

Durante la sesión, Claude fue corregido **3 veces** con "acordate de chequear X antes de Y".

### Prompt:

| “ "Esto ya te lo repetí 3 veces. ¿Lo convertimos en skill?" ”

**Skill invocada:** `superpowers:writing-skills`  
(alternativa del marketplace: el plugin `skill-creator`)

Output: un `SKILL.md` nuevo en `.claude/skills/`.

La slide del principio terminaba:

"Mañana — sesión nueva. Re-explico el ticket, el plan, las convenciones."

**Mañana — sesión nueva. No se re-explica ni el ticket, ni el plan, ni las convenciones.**

**Este es el patrón completo en acción.**



## ■ Y no siempre tenés que contar vos

Hoy lo repetí 3 veces y lo noté. Cuando no lo notás, se lo preguntás:

“ *Revisá las últimas conversaciones y decime qué skills hay que crear o actualizar.* ” ”



El patrón es el mismo — cambia quién lo detecta.

## ■ Mi línea de tiempo real

|           |          |                                                                                          |
|-----------|----------|------------------------------------------------------------------------------------------|
| semana 1  | ● BASE   | CLAUDE.md inicial: 5 líneas — nombre, comando de test, convención de commits             |
| semana 1  | ● MEMORY | feedback-local-build-before-ci — "acordate del typecheck", anotado. Y me lo olvidé igual |
| semana 1  | ● SKILL  | local-build-gate — CI roto 2 veces por un typo que typecheck cazaba en 2 seg             |
| semana 3  | ● RULE   | no-parallel-ci — los 90 minutos cazando flaky tests fantasma en stage                    |
| semana 3  | ● RULE   | english-only — un compañero no podía revisar un commit en español                        |
| semana 6  | ● SKILL  | ci-failure-triage — las mismas 3 preguntas cada lunes                                    |
| semana 6  | ● SKILL  | known-issues-registry-update — perdía el registro de qué flaky test ya vi                |
| semana 9  | ● SKILL  | multi-agent-pr-review + plugin pr-review-toolkit — de secuencial a paralelo              |
| semana 11 | ● HOOK   | typecheck-after-edit — memory + skill no alcanzaban                                      |
| semana 13 | ● SKILL  | ticket-coverage-gap-analysis — la misma conversación 4 sprints seguidos                  |
| semana 16 | ● MEMORY | known-issues — el registry crece con cada flaky test: se lee cuando hace falta           |
| ahora     | ●        | El setup quedó versionado en el repo — los compañeros lo mejoran con sus propias PRs     |
| futuro    | ○        | Nuevas ideas, nuevas necesidades — seguimos buscando patrones                            |

**Nada se planificó. Cada pieza respondió a un dolor concreto.**

## ■ El agente también construye

Cultivar no es solo skills, rules y memory **para** el agente.

Es lo que el agente construye **con vos**:

- **CI a tu medida** — el pipeline corre con tus reglas, no con las heredadas
- **Linters / checkstyle / SonarQube** — configurados y explicados, no copiados de un gist
- **Coverage y notificaciones** — Allure, dashboards, el build roto te encuentra a vos

El hook de typecheck que viste hace rato **es exactamente este patrón** — aplicado a un linter o a un quality gate, la conversación es la misma.

“ **No le tengas miedo a lo desconocido: el costo de aprender colapsó.**

Cultivás al agente — y el proyecto queda mejor armado que antes. ”

# ■ El modelo se equivoca

No es determinístico, y no avisa: **suenan igual de convincente cuando acierta que cuando no.** Cuatro formas de fallar, las cuatro vistas en mi propio setup:

| SE EQUIVOCA ASÍ                                                                                                                     | QUÉ LO AGARRA                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Inventa</b> — un flag, un endpoint, un método que no existe                                                                      | Que el resultado sea <b>verificable</b> : el hook corre typecheck, el test pasa o no pasa              |
| <b>Toma un atajo razonable</b> — le pedís tests y los escribe mirando el código del dev: pasan por construcción, con el bug adentro | Una <b>rule</b> que lo prohíbe y un <b>skill</b> que fija el orden: primero el AC, el código después   |
| <b>Se pasa de límite</b> — triggea el build que ya estaba corriendo, intenta leer credenciales del keychain                         | La <b>rule</b> lo frenó en la Demo 3. Y lo que no se negocia no se le pide: se bloquea en los permisos |
| <b>Se olvida</b> — sesión nueva, cero contexto, todo de nuevo                                                                       | <b>Memory</b> — lo que sobrevive al <code>/clear</code>                                                |

**Mi trabajo no es que no se equivoque. Es que cuando pase, se vea — y que el mismo error no vuelva dos veces.**

“ Cuando aparece uno nuevo no lo corrijo en el chat y sigo de largo: **lo escribo.** Así nació cada pieza que vieron hoy. ”

■ Qué le toca a la persona

|                 | EL AGENTE                                                                         | VOS                                                                                               |
|-----------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Ejecución       | Corre el procedimiento, agrega resultados, propone cambios de código para revisar | —                                                                                                 |
| Criterio        | —                                                                                 | Interpretás un AC ambiguo, decidís qué es "suficiente"                                            |
| Aporte          | —                                                                                 | Metés lo que el agente no vio: charlas, discusiones, decisiones de negocio — enriquecés, corregís |
| Revisión        | Hace el primer pase (5 subagentes en paralelo)                                    | Leés el resultado antes de postearlo o mergear                                                    |
| Decisión        | —                                                                                 | Aprobás, bloqueás, o decidís qué se promueve a skill/rule                                         |
| Responsabilidad | —                                                                                 | Si preguntan por qué, la respuesta sos vos                                                        |

**El agente ejecuta. La persona aporta lo que falta, decide, revisa y responde por el resultado — eso no se delega.**

*Mismo criterio que en la Demo 4: se lee entero antes de postearlo — "no sé, lo hizo la IA" no es una respuesta válida.*

La pregunta del principio

**¿Puedo automatizar mi proceso de QA con IA?**

**Sí — lo acabás de ver.**

**¿Se diseña? No. Se cultiva.**

## ■ 4 pasos para el lunes

**1. Un `CLAUDE.md` de 5 líneas** en el repo donde más trabajás.

Nombre del proyecto, comando de test, convención de commits. Nada más.

**2. Probá hacer todo con el agente, aunque hoy lo hagas por fuera.**

Leer el ticket de Jira, estudiar la story, compararla contra el código.

Ahí empiezan a aparecer los patrones repetibles.

**3. Anotá la próxima corrección que repitas.**

La segunda vez que escribís "*acordate de X antes de Y*", eso es una memory.

La tercera, un skill.

**4. Bajá lo que ya existe:** `/plugin marketplace add` y después `/plugin install` → `superpowers` + `pr-review-toolkit`.

Arrancás con workflows que no tuviste que escribir. Son públicos: usá los de confianza, los que tu organización ya aceptó.

| “ Nada de esto necesita presupuesto, permiso, ni una reunión de arquitectura. ”

# Tu setup no se diseña, se cultiva.

**Memory + Skills + Rules + Hooks = workflow reproducible**

**Cultivarlo no te saca del medio: el criterio, el dominio y la firma siguen siendo tuyos.**

¿Preguntas?

[github.com/edcrove/claude-qa-demo](https://github.com/edcrove/claude-qa-demo)

[linkedin.com/in/edgardocrovetto](https://linkedin.com/in/edgardocrovetto)



# Apéndice

Anatomía y ejemplo real de cada pieza

Memory · Skill · Rule · Hook

## ■ Memory — anatomía

Una corrección repetida **2 veces** es candidata a memory.

### Prompt:

“ *Guardá esto en memory: <la corrección o el hecho>.* ”

```
---
name: kebab-case-slug
description: <One-line summary; Claude lo usa para decidir si traer este recuerdo>
metadata:
  type: feedback | user | project | reference
---

<Body: el hecho, la preferencia, la corrección>

**Why:** <razón concreta – usualmente un incidente pasado>
**How to apply:** <cuándo aplica>
```

Ese prompt es lo único que hace falta — Claude arma el archivo. Persiste entre sesiones. Sobrevive al `/clear`.

## ■ Memory — ejemplo real

### Prompt:

“ *Guardá esto en memory: corré typecheck y tests locales antes de cualquier build remoto. Me olvidé 2 veces después de un rebase.* ”

```
---
name: feedback-local-build-before-ci
description: Always run local typecheck + tests before triggering remote CI
metadata:
  type: feedback
---
```

Run `npm run typecheck && npm test` before any remote CI build.

**\*\*Why:\*\*** failed CI runs burn ~10 minutes per attempt.

**\*\*How to apply:\*\*** see `local-build-gate` skill.

Nació después de olvidarme el `typecheck` post-rebase un lunes a la mañana.

## ■ Skill — anatomía

### Prompt:

“ *Convertí esto en un skill.* ”

```
---
name: kebab-case-name
description: Use when <trigger>. <One-sentence summary.>
---

# Title

## When to use
- Concrete trigger 1
- Concrete trigger 2

## Steps
1. Concrete and verifiable
2. ...

## Output
What the skill produces.
```

**description** es el campo crítico: Claude lo usa para decidir si invocar.

## ■ Skill — ejemplo real

### Prompt:

“ *"Rompi el build dos veces esta semana por lo mismo. Convertí el chequeo local en un skill."* ”

```
---
name: local-build-gate
description: Use before triggering remote CI to fail fast on
  compile or unit-test errors. Saves ~10 min per broken push.
---

# Local build gate

1. **Typecheck:** cd demo-app && npm run typecheck
2. **Unit tests:** cd demo-app && npm test
3. **Smoke check:** call the endpoint once

## When to skip
Never. Compose with `no-parallel-ci.mdc`.
```

Nació después de romper el build dos veces en la misma semana.

## ■ Rule — anatomía

### Prompt:

“ *Convertí esto en una rule — quiero que esté siempre presente.* ”

```
---  
description: One-line summary, ≤ 120 chars, present tense  
---
```

Rule statement: what you must / must not do.

**\*\*Why:\*\*** the reason – often a past incident.

**\*\*How to apply:\*\*** when this kicks in.

A diferencia de un skill, **siempre está cargada** — pero no por ser una rule: lo está porque `CLAUDE.md` la **importa** con `@`. Sin ese import, el archivo no hace nada, y su `description` —a diferencia de la de un skill— no la lee nadie.

## ■ Rule — ejemplo real

### Prompt:

“ *"Esto no puede volver a pasar. Regla: nunca triggerear un build si ya hay otro corriendo en el mismo ambiente."* ”

```
---  
description: Never trigger two CI builds in parallel against  
  the same TEST_ENV – they share credentials.  
---  
  
Before triggering a CI build:  
1. Check whether another build is already running on the env.  
2. If yes, wait or pick a different env. Do not queue parallel.  
  
**Why:** parallel runs share credentials → mutual interference  
that looks like real regressions but isn't.
```

Nació después de 90 minutos cazando flaky tests fantasma.

## ■ Hook — anatomía

### Prompt:

“ *Convertí esto en un hook — que corra solo, sin que nadie lo invoque.* ”

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write|MultiEdit",
      "hooks": [{
        "type": "command",
        "command": "<bash command>"
      }]
    }]
  }
}
```

Se engancha a momentos del ciclo: antes ( **PreToolUse** ) y después ( **PostToolUse** ) de cada tool, cierre de sesión ( **Stop** ), entre otros.

El comando recibe el evento como **JSON por stdin**.

**No depende del modelo:** lo dispara el runtime de Claude Code y siempre se ejecuta.



## ■ Hook — ejemplo real

### Prompt:

“ *Me olvidé el typecheck 5 veces seguidas — teniendo memory y skill. Convertilo en hook: que corra en cada edit, pase lo que pase.* ”

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write|MultiEdit",
      "hooks": [{
        "type": "command",
        "command": "f=$(jq -r '.tool_input.file_path // empty');
                    case \"$f\" in \"$CLAUDE_PROJECT_DIR\"/*.*ts)
                      cd \"$CLAUDE_PROJECT_DIR/demo-app\" && npm run typecheck;;
                    esac"
      }]
    }]
  }
}
```

El path sale del JSON de stdin con `jq` — no de una variable de entorno.

Nació después de olvidarme el `typecheck` 5 veces seguidas — **teniendo el memory y el skill**. El hook fue el final del camino.

## **Backup — Q&A**

Respuestas que suelen hacer falta

## ■ ¿Y los costos en tokens?

- El día a día corre en un modelo mid-tier; el multi-agent review es el único paso caro
- 5 subagentes = 5 contextos aislados — pagás tokens para **no** pagar contexto contaminado
- El costo contra el que se compara: 10 min de CI roto · 90 min de flaky tests fantasma · un review que espera 2 días
- **Lo siempre-cargado es lo único con costo recurrente:** memory y rules se pagan en cada mensaje, de cada sesión. Una skill cuesta su `description` siempre y su cuerpo sólo cuando se usa; un hook no consume contexto. Por eso las skills viven afuera hasta que hacen falta
- **Cómo se mide:** `/context` muestra qué está cargado y cuánto ocupa · `/usage` el consumo de la sesión · `/doctor` te dice **qué de eso no estás usando** · headless, `claude -p --output-format json` devuelve `total_cost_usd` por corrida
- **Evals, no intuición:** 5 tareas representativas, corridas headless con dos configuraciones (con y sin una skill, mid-tier vs. modelo grande), comparando costo y resultado. Optimizar sin medir es adivinar

## ■ ¿Qué pasa cuando cambia el modelo?

- Los **prompts** afinados a un modelo a veces mueren con él
- **Skills, rules y memory sobreviven** — describen tu proceso, no al modelo
- El **hook** ni siquiera pasa por el modelo: lo ejecuta el runtime
- Por eso la pirámide promociona hacia arriba: **cada nivel es más robusto al cambio**
- Cambiar de modelo —o de LLM— **no es migrar, es revisar**: que las estructuras sigan siendo las óptimas es una conversación con el agente nuevo. "*Optimizá estas skills y rules para cómo funcionás vos*" aplica igual en Claude, ChatGPT o Gemini

## ■ ¿Funciona offline? ¿Sirve en mi stack?

- Este demo corre **100% offline**: mocks JSON en disco, cero credenciales
- Solo los **MCPs** reales (Jira, TestRail, Jenkins, GitHub, Confluence) necesitan red
- El patrón no sabe de lenguajes: idéntico en **Java/TestNG/RestAssured**
- Todo es texto plano: clonalo y reemplazá los mocks por tus sistemas
- Lo genérico puede vivir **en un repo aparte y compatible** — skills, rules y hooks que no son de un proyecto puntual — y clonarse en cada equipo. El mío está separado del repo del producto

## ■ ¿Quién audita al clasificador de CI?

- `ci-failure-triage` no decide a ciegas: la categoría sale de evidencia visible (firma en el registry, stack, commits) — no es una caja negra
- El registry ( `memory/known-issues.md` ) lo propone el agente y lo confirmás vos antes de commitear, y pide **2+ sightings** — un solo fallo nunca es un flaky test
- Si la categoría no cierra con evidencia, **regresión real es el default** — el skill sesga hacia flaggear de más, no de menos
- Igual que en la Demo 4: el agente propone la categoría, vos la confirmás antes de actuar

## ■ Lo que no entró en la charla

La pirámide es el piso, no el techo. Otras piezas que hoy no demostré:

- **/loop** — repetir un prompt o un skill cada X minutos, o dejar que el agente marque el ritmo: mirar un build hasta que termine
- **/goal** — fijás una condición de fin y el agente sigue trabajando hasta que **un evaluador aparte** confirma que se cumplió; no es el mismo agente declarándose listo
- **/schedule** — agentes en cron: el triage de la regression nocturna ya hecho cuando llegás a la mañana
- **Background agents + worktrees** — trabajo largo sin bloquear la sesión, y cada ticket en su propio workspace aislado
- **/doctor** — te audita el setup: qué skills y MCPs pagás en contexto y no usás, qué hooks están lentos, y qué de tu **CLAUDE.md** siempre-cargado puede pasar a on-demand. Reporta primero y pregunta antes de tocar nada
- **Unattended agents** — sin nadie mirando: headless en CI, disparado por un webhook. Acá la pregunta del cierre incomoda: si nadie lo leyó, ¿quién firma?
- **Agentes propios** — un agente no es "otro Claude": se le define su rol, sus tools y su criterio. Escribir uno es un archivo de markdown

**Ninguna reemplaza a la pirámide: la usan.**

# ■ Qué abrir cuando clones el repo

| PATH                                | QUÉ ES                                                                                             |
|-------------------------------------|----------------------------------------------------------------------------------------------------|
| CLAUDE.md                           | lo que se carga en <b>toda</b> sesión: convenciones + los @ imports                                |
| .claude/rules/ ·<br>.claude/skills/ | 3 rules siempre cargadas · 6 skills que cargan cuando hacen falta — una es cómo se edita este deck |
| .claude/settings.json               | el hook de typecheck                                                                               |
| memory/                             | lo que sobrevive al /clear + el registry de known issues                                           |
| skill-templates/                    | plantillas vacías: tu primer skill, rule y hook                                                    |
| mocks/                              | Jira, Jenkins y GitHub falsos — corre sin red                                                      |
| evolution-timeline.md               | cómo creció, semana por semana y con qué dolor                                                     |
| docs/showable-inventory.md          | el catálogo de piezas ↓                                                                            |

**docs/showable-inventory.md** — catálogo de mi setup **de trabajo real**: rules, skills, agentes, hooks, permisos. Cada fila dice qué hace la pieza y **por qué existe** — el dolor que la hizo aparecer. Lo que ya está acá, va marcado.

**Cómo usarlo:** busca tu dolor en la columna "*por qué existe*" → copió la pieza → reemplazó los nombres genéricos ( `api-tests` , `PROJ-1234` ) por los tuyos. Empezá por la tabla "**si copiás sólo tres cosas**".